

Module 1 – React Fundamentals

1. React Basics Refresher

What is React?

React.js is a JavaScript library used to build fast and interactive user interfaces, especially for single-page applications (SPAs).

Key Features of React:

- Component-based architecture
- Virtual DOM for fast rendering
- Reusable UI components
- Declarative programming style

How React Works:

- React creates a **Virtual DOM**
- Compares it with the real DOM (Diffing)
- Updates only changed parts (Reconciliation)

Example:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

2. JSX Deep Dive

What is JSX?

JSX stands for **JavaScript XML**.
It allows writing HTML inside JavaScript.

Why JSX?

- Makes UI code readable
- Combines logic + UI in one place

Rules of JSX:

- Must return a single parent element
- Use `className` instead of `class`
- JavaScript inside `{ }`

Example:

```
const name = "Ali";  
const element = <h1>Hello, {name}</h1>;
```

3. Components & Props

Components

Components are reusable building blocks of a React application.

Types of Components:

- Functional Components (Modern)
- Class Components (Older)

Example:

```
function Welcome() {  
  return <h2>Welcome User</h2>;  
}
```

Props (Properties)

Props are used to pass data from parent to child components.

Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
// Usage  
<Welcome name="Ali" />
```

Key Points:

- Props are **read-only**
 - Help make components reusable
-

4. State & Lifecycle

What is State?

State is used to store dynamic data inside a component.

useState Hook:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Component Lifecycle (Simplified)

- Mounting → Component created
- Updating → State/props change
- Unmounting → Component removed

(Handled using hooks like `useEffect` in modern React)

5. Event Handling

What is Event Handling?

React handles user actions like clicks, typing, etc.

Example:

```
function ClickButton() {
  function handleClick() {
    alert("Button Clicked");
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

Key Points:

- Events use camelCase (`onClick`)
 - Pass functions, not function calls
-

6. Key Concepts to Remember

- React uses **components** to build UI
- JSX allows writing HTML in JavaScript
- Props pass data between components
- State manages dynamic data
- Events handle user interaction
- Virtual DOM improves performance

7. Summary

React is a powerful front-end library that simplifies UI development through reusable components and efficient rendering. By understanding JSX, components, props, state, and event handling, you build a strong foundation for advanced React concepts.